

Hashing — All Questions with Separate Bengali and English Answers

প্রতিটি টপিকের জন্য আগে বাংলায় প্রশ্ন ও উত্তর, তারপর English question and answer দেওয়া হলো। যেখানে সম্ভব, ছোট উদাহরণও যোগ করা হয়েছে।

১) হ্যাশিং কী?

বাংলায় উত্তর: হ্যাশিং হলো এমন একটি কৌশল, যেখানে ইনপুট ডেটাকে একটি হ্যাশ ফাংশন ব্যবহার করে নির্দিষ্ট দৈর্ঘ্যের একটি মান (hash value) এ রূপান্তর করা হয়। এই মান ব্যবহার করে ডেটা দ্রুত সংরক্ষণ ও অনুসন্ধান করা যায়।

উদাহরণ: “Rahim” → hash value 42 → hash table-এর index 42-এ সংরক্ষণ।

English Answer: Hashing is a technique that transforms input data into a fixed-size value (hash value) using a hash function. This value is used for fast storage and lookup.

Example: “Rahim” → hash value 42 → stored at index 42 in a hash table.

২) হ্যাশ ডেটা স্ট্রাকচারের প্রয়োজন কেন?

বাংলায় উত্তর:

১. দ্রুত অনুসন্ধান (Search)
২. দ্রুত সন্নিবেশ (Insert)
৩. দ্রুত অপসারণ (Delete)
৪. কী-ভিত্তিক ডেটা ব্যবস্থাপনা (key-value mapping)
৫. বড় ডেটাসেটে কার্যকর পারফরম্যান্স

উদাহরণ: ডিকশনারি, ম্যাপ, ক্যাশে, ইউজার সেশন স্টোর।

English Answer:

১. Fast search
২. Fast insertion
৩. Fast deletion
৪. Efficient key-value mapping
৫. Good performance on large datasets

Examples: Dictionaries, maps, caches, session stores.

৩) হ্যাশিংয়ের উপাদান (Components of Hashing)

বাংলায় উত্তর:

1. Key (কী): যে ডেটা দিয়ে অনুসন্ধান করা হয়।
2. Hash Function: কী থেকে hash value তৈরি করে।
3. Hash Value: ফাংশনের আউটপুট।
4. Hash Table: যেখানে ডেটা সংরক্ষণ করা হয়।
5. Collision Resolution Mechanism: সংঘর্ষ সামলানোর পদ্ধতি।

English Answer:

1. Key: Data used for lookup.
2. Hash Function: Produces a hash value from the key.
3. Hash Value: Output of the hash function.
4. Hash Table: Storage structure.
5. Collision Resolution Mechanism: Technique to handle collisions.

৪) হ্যাশিং কিভাবে কাজ করে?

বাংলায় উত্তর:

1. একটি কী ইনপুট হিসেবে নেওয়া হয়।
2. হ্যাশ ফাংশন কী-কে hash value-তে রূপান্তর করে।
3. সেই মান ব্যবহার করে টেবিলের index নির্ধারণ করা হয়।
4. ডেটা সেই index-এ সংরক্ষণ বা খোঁজা হয়।

উদাহরণ: table size = 10, key = 37, $h(k)=k \bmod 10 \Rightarrow 7$; ডেটা index 7-এ রাখা হবে।

English Answer:

1. Take a key as input.
2. Apply the hash function to get a hash value.
3. Use the value to determine a table index.
4. Store or search the data at that index.

Example: table size = 10, key = 37, $h(k)=k \bmod 10 \Rightarrow 7$; store at index 7.

৫) হ্যাশ ফাংশন কী?

বাংলায় উত্তর: হ্যাশ ফাংশন হলো এমন একটি অ্যালগরিদম, যা ইনপুট ডেটাকে একটি নির্দিষ্ট আকারের hash value-তে রূপান্তর করে।

উদাহরণ: $h(k)=k \bmod m$

English Answer: A hash function is an algorithm that converts input data into a fixed-size hash value.

Example: $h(k)=k \bmod m$

৫.a) Types of Hash Functions

বাংলায় উত্তর:

ধরন	ব্যাখ্যা
Division Method	$h(k)=k \bmod m$
Multiplication Method	গুণ করে fractional অংশ ব্যবহার করা হয়
Mid-Square Method	k^2 করে মাঝের অংশ নেওয়া হয়
Folding Method	সংখ্যাকে ভাগ করে যোগ করা হয়
Universal Hashing	একাধিক ফাংশন থেকে র্যান্ডমভাবে নির্বাচন

English Answer:

Type	Description
Division Method	$h(k)=k \bmod m$
Multiplication Method	Multiply and use the fractional part
Mid-Square Method	Square the key and take middle digits
Folding Method	Split and add parts of the key
Universal Hashing	Randomly choose from a family of hash functions

৫.b) ভাল হ্যাশ ফাংশনের বৈশিষ্ট্য

বাংলায় উত্তর:

1. দ্রুত গণনাযোগ্য
2. সমানভাবে বিতরণ করে (uniform distribution)
3. কম collision সৃষ্টি করে
4. ইনপুটে ছোট পরিবর্তনে ভিন্ন আউটপুট দেয়
5. নির্ধারক (deterministic)

English Answer:

1. Fast to compute
2. Uniform distribution
3. Few collisions
4. Small input changes produce different outputs
5. Deterministic

৫.c) Complexity of calculating hash value

বাংলায় উত্তর: সাধারণত $O(1)$ ধরা হয়, কারণ হ্যাশ ফাংশন নির্দিষ্ট সংখ্যক ধাপে কাজ করে। তবে খুব বড় ইনপুট বা জটিল ফাংশনের ক্ষেত্রে সময় ইনপুটের আকারের ওপর নির্ভর করতে পারে।

English Answer: Usually considered $O(1)$ because the function performs a fixed amount of work. For very large inputs or complex functions, the cost may depend on input size.

৬) হ্যাশিংয়ের সমস্যা: Collision কী?

বাংলায় উত্তর: যখন দুটি ভিন্ন কী একই hash value বা একই index তৈরি করে, তখন তাকে collision বলে।

উদাহরণ: table size = 10; $17 \bmod 10 = 7$ এবং $27 \bmod 10 = 7 \Rightarrow$ collision।

English Answer: A collision occurs when two different keys produce the same hash value or table index.

Example: table size = 10; $17 \bmod 10 = 7$ and $27 \bmod 10 = 7 \Rightarrow$ collision.

৬.a) Collision কীভাবে সামলানো হয়?

বাংলায় উত্তর: দুটি প্রধান পদ্ধতি:

1. Separate Chaining
2. Open Addressing (Linear, Quadratic, Double Hashing)

English Answer: Two main approaches:

1. Separate Chaining
2. Open Addressing (Linear, Quadratic, Double Hashing)

৬.a.১) Separate Chaining

বাংলায় উত্তর: প্রতিটি index-এ একটি linked list বা chain রাখা হয়। Collision হলে একই index-এর চেইনে নতুন আইটেম যোগ করা হয়।

উদাহরণ: index 7 $\rightarrow$ [17] $\rightarrow$ [27] $\rightarrow$ [37]

English Answer: Each table index stores a linked list (chain). On collision, the new item is appended to the chain at that index.

Example: index 7 $\rightarrow$ [17] $\rightarrow$ [27] $\rightarrow$ [37]

৬.a.২) Open Addressing

বাংলায় উত্তর: Collision হলে টেবিলের অন্য খালি slot খুঁজে সেখানে ডেটা রাখা হয়।

English Answer: On collision, another empty slot in the table is searched and used for storage.

৬.১.২.১) Linear Probing

বাংলায় উত্তর: পরপর slot পরীক্ষা করা হয়।

$$(h(k)+i) \bmod m$$

উদাহরণ: 7 ভরা থাকলে 8, তারপর 9, তারপর 0...

English Answer: Check the next slots sequentially.

$$(h(k)+i) \bmod m$$

Example: If slot 7 is occupied, try 8, then 9, then 0...

৬.১.২.২) Quadratic Probing

বাংলায় উত্তর: Quadratic দূরত্বে slot পরীক্ষা করা হয়।

$$(h(k)+i^2) \bmod m$$

উদাহরণ: 7, 8, 11, 16 ... (mod m)

English Answer: Probe slots at quadratic distances.

$$(h(k)+i^2) \bmod m$$

Example: 7, 8, 11, 16 ... (mod m)

৬.১.২.৩) Double Hashing

বাংলায় উত্তর: দ্বিতীয় একটি hash function ব্যবহার করে step size নির্ধারণ করা হয়।

$$(h_1(k)+i \cdot h_2(k)) \bmod m$$

উদাহরণ: $h_1(k)=k \bmod 10$, $h_2(k)=7-(k \bmod 7)$

English Answer: Use a second hash function to determine the probe step size.

$$(h_1(k)+i \cdot h_2(k)) \bmod m$$

Example: $h_1(k)=k \bmod 10$, $h_2(k)=7-(k \bmod 7)$

৭) Load Factor in Hashing

বাংলায় উত্তর: Load factor নির্দেশ করে টেবিল কতটা ভর্তি হয়েছে।

$$\alpha = n/m \quad \alpha = n / m$$

যেখানে n = উপাদানের সংখ্যা, m = টেবিলের সাইজ। α বেশি হলে collision বাড়ে।

English Answer: Load factor indicates how full the table is.

$$\alpha = n/m \quad \alpha = n / m$$

where n = number of elements, m = table size. A higher α usually increases collisions.

৮) Rehashing কী?

বাংলায় উত্তর: টেবিল খুব ভরে গেলে (load factor threshold ছাড়ালে) নতুন বড় টেবিল তৈরি করে সব উপাদান আবার hash করে স্থানান্তর করা হয়।

উদাহরণ: size 10 $\rightarrow$ size 20; সব key নতুন করে index গণনা করে রাখা হয়।

English Answer: When the table becomes too full (load factor exceeds a threshold), create a larger table and hash all elements again into the new table.

Example: size 10 $\rightarrow$ size 20; recompute indices for all keys.

৯) Applications of Hash Data Structure

বাংলায় উত্তর:

1. Dictionary / Map
2. Set implementation
3. Database indexing
4. Symbol table (compiler)
5. Cache lookup
6. Duplicate detection

English Answer:

1. Dictionary / Map
2. Set implementation
3. Database indexing
4. Symbol tables in compilers
5. Cache lookup
6. Duplicate detection

১০) Real-Time Applications

বাংলায় উত্তর:

1. Web browser cache
2. Redis / Memcached
3. Database hash indexes
4. Password verification (salted hashing)
5. Content-addressable storage (e.g., Git object IDs)
6. Session management in web apps

English Answer:

1. Web browser cache
2. Redis / Memcached
3. Database hash indexes
4. Password verification (salted hashing)
5. Content-addressable storage (e.g., Git object IDs)
6. Session management in web applications

১১) Advantages of Hash Data Structure

বাংলায় উত্তর:

1. Average-case search, insert, delete $\approx O(1)$
2. দ্রুত key-based lookup
3. বড় ডেটাসেটে ভালো পারফরম্যান্স
4. ডিকশনারি ও সেট বাস্তবায়নে উপযোগী

English Answer:

1. Average-case search, insert, delete $\approx O(1)$
2. Fast key-based lookup
3. Good performance on large datasets
4. Ideal for dictionaries and sets

১২) Disadvantages of Hash Data Structure

বাংলায় উত্তর:

1. Collision সমস্যা
2. খারাপ hash function হলে পারফরম্যান্স কমে যায়
3. Worst case $O(n)$
4. ক্রম (ordering) বজায় থাকে না

5. Rehashing ব্যবহৃত হতে পারে
6. টেবিল সাইজ ও load factor টিউনিং প্রয়োজন

English Answer:

1. Collision issues
2. Poor hash functions can degrade performance
3. Worst-case $O(n)$
4. No inherent ordering of elements
5. Rehashing can be expensive
6. Requires tuning of table size and load factor

Exam-ready one-line summary

বাংলা: “হ্যাশিং হলো এমন একটি কৌশল যেখানে একটি hash function ব্যবহার করে key-কে table index-এ রূপান্তর করা হয়, ফলে average case-এ search/insert/delete প্রায় $O(1)$ সময়ে করা যায়; তবে collision মোকাবিলার জন্য chaining বা open addressing প্রয়োজন হয়।”

English: “Hashing is a technique that uses a hash function to map keys to table indices, enabling average-case $O(1)$ search/insert/delete operations, while collisions are handled using chaining or open addressing.”